

Diagnosis-Driven Optimization for Clickbait Spoiler-Type Classification and Generation

Rocha Shao and Xiaohui Li

University of Waterloo

{r9shao, x2436li}@uwaterloo.ca

Abstract

Clickbait spoiling requires predicting the form of a missing answer and generating that answer from a linked article. For Task 1 (spoiler-type classification), we compared RoBERTa and two DeBERTa generations at base and large scale. Both DeBERTa generations favored their base variants, and targeted fine-tuning of DeBERTa-v1-base with class weighting, FGM adversarial training, and cosine scheduling reached a Kaggle Macro-F1 of **0.77304**. For Task 2 (spoiler generation), Flan-T5-large was the strongest generator tested but plateaued near 0.41 METEOR; oracle evidence experiments revealed that `passage` type had massive improvement potential (+0.27 METEOR with perfect evidence), identifying evidence selection as the main bottleneck rather than generation capacity. A pipeline combining diverse candidate generation, Token-F1 consensus selection, and passage-specific continuation training reached a final Kaggle test METEOR of **0.47303**. These results show that model scale was task-dependent, and that targeted pipeline redesign based on systematic error diagnosis was more important than capacity alone. Code is available at: <https://github.com/HaileyLiData/MSE641-Clickbait-Spoiler-Detection>

1 Introduction

Clickbait posts create a curiosity gap by withholding the key information needed to satisfy a reader’s interest, encouraging a click to a linked article. Clickbait spoiling aims to close this gap automatically: identifying what form of information is missing and generating a concise spoiler from the article.

This project addresses both tasks:

- **Task 1:** Classify each example into one of three spoiler types — `phrase`, `passage`, or `multi`

- **Task 2:** Generate the corresponding spoiler text

Task 1 is a small (~3,200-example), imbalanced three-class classification problem — `multi` is fewer than half as many examples as either other class — whereas Task 2 must generate outputs with substantially different lengths and structures, and correctly identifying *which* fact resolves the post is a harder requirement than producing an article-relevant, fluent output.

Our approach follows a **diagnosis-driven optimization** strategy. For Task 1, comparisons across RoBERTa and two DeBERTa generations showed that both DeBERTa generations favored their base variants; we therefore fixed DeBERTa-v1-base and applied class weighting, FGM adversarial training, and cosine scheduling, reaching Kaggle Macro-F1 **0.77304**. For Task 2, after generator-only tuning plateaued, **oracle evidence experiments** revealed that `passage` type had massive improvement potential (+0.27 METEOR), identifying evidence selection rather than generation capacity as the main bottleneck. This insight led us to reframe the problem from single-output generation to candidate generation and consensus-based selection, producing the final Kaggle test METEOR of **0.47303**.

2 Related Work

2.1 Spoiler-Type Classification

Hagen et al. introduced clickbait spoiling together with the Webis Clickbait Spoiling Corpus 2022 (Hagen et al., 2022), first predicting the spoiler type and then generating each type’s spoiler with a dedicated method.

For encoder-based classification, RoBERTa (Liu et al., 2019) provides a strong pretrained baseline via robust masked-language-model pre-training on a larger corpus than the original BERT,

while DeBERTa (He et al., 2021) introduces disentangled attention (separating content and position representations) and, in its v3 revision, replaces masked-language-model pretraining with a replaced-token-detection objective. On the fine-tuning side, FGM adversarial training (Miyato et al., 2017) perturbs the embedding layer along the gradient direction during training to improve local robustness, and class-weighted loss is a standard remedy for the kind of class imbalance present in this dataset’s `multi` category.

2.2 Spoiler Generation

Hagen et al.’s original system treated `phrase` spoiling as extractive question answering, following the span-prediction formulation popularized by SQuAD (Rajpurkar et al., 2016), and `passage` spoiling as passage retrieval; their finding that single-span extraction does not naturally extend to multipart answers is consistent with the multi-span sequence-tagging approach of Segal et al. (Segal et al., 2020), developed for larger extractive datasets than this project’s `multi` examples.

On the generative side, our formulation builds on T5 (Raffel et al., 2020), an encoder-decoder pretrained as a unified text-to-text model across tasks, and its instruction-tuned variant Flan-T5 (Chung et al., 2022), further tuned on a large mixture of instruction-phrased tasks. Sampling-based minimum Bayes-risk decoding (Eikema and Aziz, 2022) provides a theoretical framing for consensus-based selection, evaluating a candidate by its expected agreement with other hypotheses rather than in isolation.

3 Approach

This section describes our methodology and design choices. Detailed experimental results and comparisons are presented in Section 4.

3.1 Task 1: Classification Methodology

3.1.1 Model Selection

We compared three model families at both base and large scales: RoBERTa (Liu et al., 2019), DeBERTa-v1, and DeBERTa-v3 (He et al., 2021). Our hypothesis was that larger models would provide better performance on this classification task. However, initial experiments revealed an unexpected pattern: both DeBERTa generations showed base outperforming large, suggesting that under this small, three-class dataset, increasing

DeBERTa capacity specifically provided no benefit — a pattern compatible with an overfitting or optimization-stability explanation.

This result changed the direction of Task 1 development: instead of continuing to search for larger backbones, we fixed DeBERTa-v1-base and focused on errors specific to the dataset.

3.1.2 Targeted Fine-Tuning

Per-class error analysis on DeBERTa-v1-base showed `multi` — the least frequent class — had the weakest F1 and the lowest recall of the three, disproportionately costly under Macro-F1’s equal class weighting. We applied three targeted fine-tuning changes to address this:

- **Class-weighted cross-entropy:** Weights computed inversely proportional to class frequency, directly counteracting `multi`’s under-representation in the training set.
- **FGM adversarial training (Miyato et al., 2017):** Perturbing the embedding layer along the gradient direction during training to improve robustness to small input-representation changes.
- **Cosine learning-rate scheduling:** With warmup ratio for smoother convergence over this dataset’s small number of training steps.

3.2 Task 2: Generation Methodology

3.2.1 Formulation Selection

Task 2 generates the spoiler that resolves the curiosity gap; unlike Task 1, the output is not drawn from a fixed label set — a spoiler may be a short phrase, a full explanatory passage, or several disjoint facts. We compared extractive and generative formulations:

- **Extractive QA (Rajpurkar et al., 2016):** Performs well on gold-type subsets but degrades under predicted routing — when misrouted, the extractive model returns a confidently wrong short span, whereas the generative model degrades more gracefully.
- **BIO-tagging for multi-span `multi` type (Segal et al., 2020):** Underperforms generation due to limited usable training data after span-label construction.
- **Generative formulation (T5 family (Raffel et al., 2020)):** Provides flexibility across

all spoiler types. We selected Flan-T5-large (Chung et al., 2022) as the strongest generator.

3.2.2 Bottleneck Diagnosis via Oracle Evidence Experiments

After generator-only tuning plateaued, we designed **oracle evidence experiments** to diagnose the bottleneck: providing the model with oracle evidence (the sentence(s) containing the gold spoiler) and measuring performance improvement potential.

The results were striking: `passage` type improved from 0.35 to 0.61 METEOR (+0.27) with perfect evidence, while `phrase` showed minimal change and `multi` actually decreased. This revealed that the problem was evidence selection, not generation capacity — the model *can* generate correct answers when given the right evidence. **This finding fundamentally changed our optimization direction:** instead of improving the generator, we focused on improving evidence/candidate selection.

3.2.3 Multi-Candidate Generation + Consensus Selection

Based on the bottleneck diagnosis, we hypothesized that generating multiple candidate outputs and selecting the best one through consensus could address the evidence selection problem without requiring explicit evidence localization.

Stage 1: Diverse Candidate Generation. We designed a diverse decoding strategy combining:

- Beam search with varying beam widths
- Temperature sampling with multiple temperature values
- Diverse beam search with group penalty

Stage 2: Token-F1 Consensus Selection. Rather than scoring candidates independently by length and lexical overlap (which can select topically related but factually wrong candidates), we designed a consensus-based selector: select the candidate with the highest average pairwise Token-F1 similarity to the rest of the pool,

$$S(c_i) = \frac{1}{N-1} \sum_{j \neq i} \text{TokenF1}(c_i, c_j), \quad (1)$$

a lexical analogue of minimum Bayes-risk decoding (Eikema and Aziz, 2022).

3.2.4 All-F1 Extension

Initially, we applied Token-F1 consensus only to `passage` type (the identified bottleneck). We then hypothesized that extending consensus selection to all three types (**All-F1**) might provide additional gains, as the consensus principle should generalize across spoiler types.

3.2.5 Passage Specialist Training

We further hypothesized that continuation fine-tuning on passage-only data might improve passage performance while retaining performance on other types. This **Passage Specialist** approach trains from a base checkpoint and specializes it for the hardest type. The complete pipeline design is presented in Section 4.2.

4 Experiments

4.1 Task 1 Experiments

4.1.1 Dataset and Setup

We use the Webis Clickbait Spoiling Corpus (Hagen et al., 2022): 3,200 labeled training examples and 400 labeled validation examples (`phrase`: 1,367/162; `passage`: 1,274/154; `multi`: 559/84), plus 400 hidden Kaggle test examples evaluated only through leaderboard submission. Task 1 classifies each example into `phrase`, `passage`, or `multi` from its post text, target title, and target article paragraphs, and is scored with **Macro-F1**, which weights all three classes equally; we use this metric specifically because `multi` is fewer than half as many examples as either other class, so accuracy alone would let strong performance on the two larger classes mask weak performance on `multi`.

Training was conducted in Google Colab using T4 or A100 GPUs, with Python 3.12, PyTorch 2.x, and Hugging Face `transformers`. Training configuration: max input length 512, learning rate $2e-5$, 5 epochs, batch size 16, AdamW optimizer, cosine LR scheduler with 0.1 warmup, FGM epsilon 0.5, class weights 0.780/0.837/1.908.

4.1.2 Model Scale Comparison

Table 1 shows Kaggle Macro-F1 for different model families and scales. (RoBERTa’s exact archived Kaggle submission scores were not preserved, so that row is reported only qualitatively.)

DeBERTa-v1-base was the strongest configuration overall. More importantly, **both DeBERTa generations showed the same scale pattern:**

Model family	Base	Large	Δ
RoBERTa	~ 0.740	~ 0.745	+0.005
DeBERTa-v3	0.74024	0.72206	-0.018
DeBERTa-v1	0.76565	0.74680	-0.019

Table 1: Task 1 architecture and scale comparison (Kaggle Macro-F1). Both DeBERTa generations show base > large.

Component	Setting
Base model	DeBERTa-v1-base
Class weights	0.780 / 0.837 / 1.908
FGM epsilon	0.5
LR scheduler	Cosine (0.1 warmup)
Learning rate	2e-5
Epochs	5
Batch size	16
Baseline $\rightarrow$ Final	0.76565 $\rightarrow$ 0.77304 (+0.007)

Table 2: Task 1 fine-tuning configuration and improvement.

base outperformed large by approximately 0.018 Macro-F1 (v1: +0.01885; v3: +0.01818), despite the large models having substantially greater capacity. RoBERTa did not reproduce this direction (large ~ 0.745 vs. base ~ 0.740), though this difference was small. We therefore do not claim that smaller encoders are generally superior; rather, under this small, three-class dataset, increasing DeBERTa capacity specifically provided no benefit.

4.1.3 Fine-Tuning Results

Table 2 shows the fine-tuning configuration and results. Adding class weighting, FGM, and cosine scheduling raised Kaggle Macro-F1 from 0.76565 to **0.77304** (+0.007). We applied these changes as a jointly motivated package targeting the weak *multi* class.

Figure 1 illustrates the final Task 1 classification pipeline.

4.1.4 Negative Results

We tested three further extensions on top of this configuration (Table 3), none of which improved on it.

Head-tail truncation lost useful middle context, while pseudo-labeling and multi-seed voting provided no reliable additional signal beyond what the single fine-tuned model already captured. **Task 1’s central finding is therefore a change in strategy:** once broader model comparison showed that increasing capacity gave no bene-

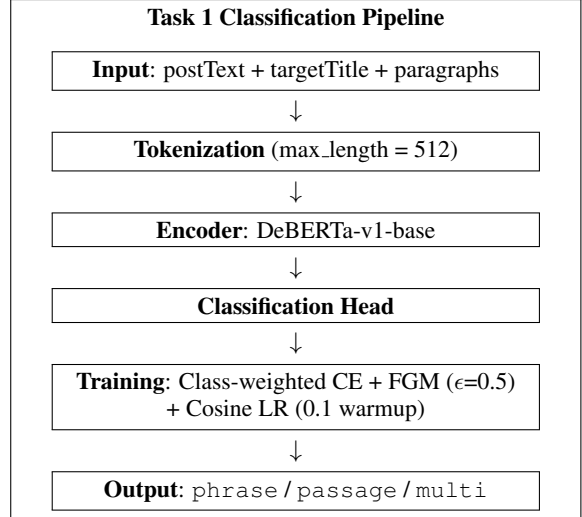


Figure 1: Task 1 classification pipeline. Final Kaggle Macro-F1: **0.77304**.

Extension	Val. F1	Outcome
Head-tail input truncation	0.7442	Lost middle context
Pseudo-labeling (≥ 0.90 conf.)	0.7508	No additional signal
5-seed soft-voting ensemble	0.7618	Did not outperform
Final model	0.7666	Kaggle: 0.77304

Table 3: Task 1 extensions evaluated on validation set. All three scored below the final model (Val. 0.7666), so were not submitted to Kaggle.

fit, we stopped searching for larger backbones and instead applied targeted, error-driven fine-tuning.

4.2 Task 2 Experiments

4.2.1 Dataset and Setup

The dataset is the same as Task 1. Task 2 is scored with **METEOR** (Banerjee and Lavie, 2005), an alignment-based metric combining unigram precision and recall (weighted toward recall) with a fragmentation penalty. Training configuration: Flan-T5-large, max input/output 512/128, learning rate 5e-5, Adafactor optimizer, 5 epochs base + 3 epochs specialist, batch size 1 with gradient accumulation 16. Training was conducted on Google Colab 16GB GPU, with ~ 1.5 –2 GPU-hours per seed.

4.2.2 Baseline and Formulation Comparison

The course-provided baseline formulates Task 2 as extractive question answering using a DeBERTa-large QA checkpoint (webis/clickbait-spoiling-with-question-answering), treating the clickbait title as the question and extracting an answer span from the article. This baseline reached **Kaggle**

METEOR 0.358.

Starting from this baseline, we explored multiple formulations to improve performance. Table 4 summarizes the exploration.

Flan-T5-large was the strongest generator evaluated, improving Kaggle METEOR from 0.358 (baseline) to 0.412 (+0.054). This is a notable contrast with Task 1, where both DeBERTa generations favored their base variants. Continued fine-tuning of the generator alone then plateaued near 0.41 Kaggle METEOR, motivating a shift toward systematic error diagnosis.

4.2.3 Per-Type Bottleneck Analysis

Breaking down METEOR by spoiler type for the base system revealed `passage` as the main bottleneck (Table 5).

`passage` was substantially weaker (0.3173) compared to `phrase` (0.5077) and `multi` (0.4603), and given its share of the data, the main drag on the overall score. This redirected the investigation from improving the generator overall to understanding why `passage` specifically was hard.

4.2.4 Oracle Evidence Experiments

To diagnose whether the bottleneck was generation capacity or evidence selection, we conducted oracle evidence experiments: providing the model with oracle evidence (the sentence(s) containing the gold spoiler) and measuring performance improvement potential.

Table 6 shows the dramatic results: `passage` type improved from 0.3456 to 0.6123 (+0.2667) with perfect evidence, while `phrase` showed minimal change and `multi` actually decreased. This revealed three key insights:

1. **Passage is the main bottleneck:** From 0.35 to 0.61 (+0.27), the largest improvement potential among all types
2. **The problem is evidence selection, not generation:** The model *can* generate correct answers when given the right evidence
3. **Multi does not benefit from oracle evidence:** Suggesting different challenges (answer structure, not localization)

This finding fundamentally changed our approach: instead of improving the generator, we focused on improving evidence/candidate selection.

4.2.5 Why Learned Retrieval Failed

We attempted to learn evidence selection directly. We trained an independent sentence classifier and a multiple-choice ranker on `passage` examples; additional retrieval/reranking variants (TF-IDF and Sentence-BERT retrieval, a cross-encoder reranker) showed the same pattern: they often recovered text about the correct topic but not the fact that resolved the clickbait question, performing close to a random baseline by this measure. Top-1 exact-match accuracy against the annotated spoiler sentence was only 34.7%; mean METEOR was 0.82 when the selected sentence matched the annotation and only 0.088 when it did not — a severe gap consistent with the models relying on topical relevance rather than exact-answer identification.

Key insight: Clickbait titles and spoiler answers have no direct lexical or semantic cues for retrieval — the relationship is pragmatic (curiosity resolution), not semantic similarity.

4.2.6 Candidate Generation and Selector Comparison

Since direct evidence localization failed, we tested whether generating multiple candidates and selecting the best one could address the problem. We generated 13 candidates per sample using diverse decoding strategies (Table 7).

On the same candidate pool, we compared two selectors (Table 8): replacing length-overlap with deduplicated Token-F1 consensus improved Kaggle METEOR from 0.417 to 0.437 (+0.020), isolating the selector as the source of improvement with the generator checkpoint, Task 1 predictions, and candidate pool all held fixed.

4.2.7 All-F1 Extension Results

We compared the original type-specific selector (Token-F1 consensus for `passage` only) with Token-F1 consensus extended to all three types (**All-F1**) on identical candidate pools across multiple seeds. All-F1 improved all five evaluated checkpoints by 0.0035–0.0181 validation METEOR (e.g. Seed 42: 0.4643 to 0.4761), indicating the gain was not specific to one seed.

4.2.8 Seed and Specialist Comparison

We trained Flan-T5-large from three random seeds (42, 123, 2026) and continued fine-tuning the top two base checkpoints on `passage`-only data (~1,274 examples). Table 9 shows the comparison.

Formulation / model	Key result	Decision
<i>Teacher's baseline</i>	DeBERTa-large QA (extractive); Kaggle 0.358	Starting point
Extractive QA (warm-started)	Strong on gold subset (0.4522); weaker under predicted routing (0.4208 vs. 0.430)	Reject
BIO multi-span tagging (<i>multi</i>)	Limited usable data (523 examples); underperformed generation	Reject
BART-base	Outputs systematically too short	Reject
T5-base	Stable generative baseline (val. 0.4257)	Retain for comparison
Flan-T5-base	Did not beat T5-base under identical recipe (0.4234 vs. 0.4257)	Reject
Flan-T5-large	Strongest generator (val. 0.4591, Kaggle 0.412)	Select

Table 4: Task 2 formulation and generator exploration. Starting from teacher’s baseline (0.358), we explored extractive and generative approaches.

Type	METEOR	Samples	Status
phrase	0.5077	162	Good
multi	0.4603	84	Moderate
passage	0.3173	154	Bottleneck
Overall	0.4257	400	–

Table 5: Per-type METEOR breakdown revealing passage as the main bottleneck.

Type	Original	Oracle-1	Oracle-2	Oracle-3
phrase	0.5676	0.5595	0.6105	0.5988
passage	0.3456	0.6123	0.5485	0.4835
multi	0.4212	0.2979	0.2766	0.3377
Overall	0.4514	0.5247	0.5165	0.4995

Table 6: Oracle evidence experiments. Oracle- k provides the k sentences containing the gold spoiler. *passage* shows massive improvement potential (+0.27), identifying evidence selection as the bottleneck.

Seed 2026 was the strongest *base* checkpoint (0.4928), but specialization reversed the ranking: *passage* improved for both seeds, yet Seed 123 also retained or improved the other two types and reached 0.5007 overall, while Seed 2026’s *multi* score dropped enough to make its specialization a net regression (0.4928 $\rightarrow$ 0.4876). **The strongest base model did not produce the strongest specialized model** — Seed 123 was retained because it responded better to *passage*-only continuation training, not because it started strongest.

4.2.9 System Progression

Table 10 summarizes the system progression, distinguishing between Kaggle submissions and local validation experiments.

Figure 2 illustrates the final end-to-end system,

Decoding Strategy	Parameters	Count
Beam search	beam $\in \{2,4,6,8\}$	4
Temperature sampling	$T \in \{0.7,0.9,1.1\} \times 2$	6
Diverse beam search	3 groups, penalty=0.5	3
Total		13

Table 7: Diverse candidate generation strategies.

Selector	Kaggle METEOR	Δ
Length + Overlap	0.417	–
Token-F1 Consensus	0.437	+0.020

Table 8: Selector comparison on the same candidate pool.

achieving validation METEOR 0.5007 and **Kaggle 0.47303**.

4.2.10 Negative Results

Pooling all five checkpoints into 65 candidates (before deduplication) for one All-F1 selection produced only +0.0002 overall validation METEOR while reducing *passage* specifically by 0.0120, so we retained the single Seed-123 specialist as the final system. **Candidate quality matters more than quantity** — weak candidates from other seeds diluted the strong ones.

4.3 Error Analysis

We inspect the final system’s validation predictions and identify three recurring failure modes, each individually verified against the actual validation data and candidate-pool files:

1. **Consensus amplifying truncation:** Consensus could amplify shared truncation: in one *passage* example, 11 of 13 candidates omitted the answer’s final payoff, so the in-

Checkpoint	Phr.	Pass.	Multi	Overall
Seed123, base	0.5564	0.4372	0.4397	0.4847
Seed123, spec.	0.5577	0.4650	0.4605	0.5007
Seed2026, base	0.5522	0.4379	0.4894	0.4928
Seed2026, spec.	0.5434	0.4451	0.4641	0.4876

Table 9: Base vs. passage-specialized checkpoints (validation METEOR, All-F1). The strongest base model (Seed2026) did not produce the strongest specialized model.

complete candidate’s consensus score (0.923) exceeded the complete answer’s (0.841) and All-F1 selected it by construction.

2. **Topically relevant but factually wrong:** Some outputs were topically relevant but answered a nearby fact rather than the one asked. Among the 32 lowest-scoring `phrase` validation examples, 30 (93.8%) were confirmed exact substrings of the source article — valid but wrong spans rather than malformed ones.
3. **Task 1 routing errors:** Task 1 routing errors constrained generation directly: for one `multi` example, Task 1 predicted `phrase`, so the generator produced only a single short answer and could not recover the three required facts.

These cases identify three remaining limitations — answer completeness, exact-fact identification among plausible alternatives, and dependence on upstream type predictions — that neither All-F1 nor the Passage Specialist fully addresses.

5 Conclusion

Task 1: Increasing DeBERTa capacity was not beneficial under this small, three-class dataset: base consistently outperformed large within both evaluated generations. After DeBERTa-v1-base was selected, targeted fine-tuning (class weighting, adversarial training, and learning-rate scheduling) addressed the weaker `multi` class and produced **Macro-F1 0.77304**; three further extensions each failed to add genuinely new information beyond it.

Task 2: Our key contribution was the **oracle evidence diagnosis**: revealing that `passage` had +0.27 METEOR improvement potential with perfect evidence, identifying evidence selection (not generation capacity) as the bottleneck. Since

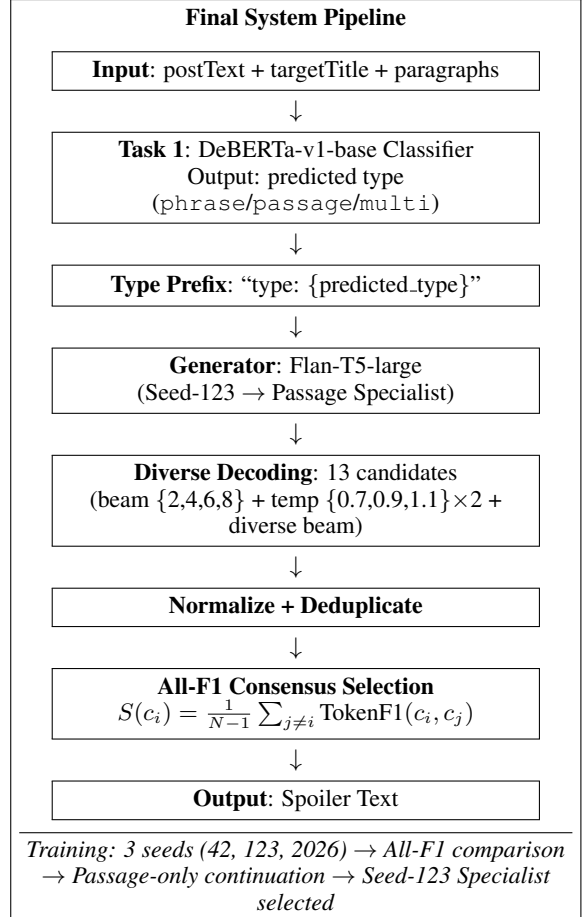


Figure 2: Final end-to-end system pipeline. Generator trained with multi-seed comparison, then continued on passage-only data.

learned retrieval failed (clickbait-spoiler relationship is pragmatic, not semantic), we reframed the problem as diverse candidate generation + consensus selection. Extending consensus to all types (All-F1) and passage-specific continuation training further improved results, achieving **METEOR 0.47303**.

Cross-Task Insights: Model scale was task-dependent (base > large for classification, but large beneficial for generation). Systematic error diagnosis was more valuable than capacity scaling. Candidate quality mattered more than quantity (5-model ensemble hurt performance).

Future Work: Combine agreement with completeness and factuality when selecting among candidates; cluster near-duplicate candidates before consensus rather than removing only exact duplicates; reduce dependence on a single predicted spoiler type; evaluate separate, type-specific generators against the current shared-generator design.

System	Kaggle	Val.	Δ
Teacher’s baseline (DeBERTa-large QA, extractive)	0.3577	–	–
Flan-T5-large, single output (our base generator)	0.4124	0.4591	+0.055
+ Type-conditioned + length-overlap (13 cand.)	0.4404	–	+0.028
+ Token-F1 consensus (passage-only)	0.4541	–	+0.014
+ All-F1 (all types) + Seed comparison	–	0.4928	–
+ Passage Specialist (Seed-123, final)	0.4730	0.5007	+0.019
Total improvement from baseline	+0.115 (+32.3%)		

Table 10: Task 2 system progression. Kaggle = submitted scores; Val. = local validation. Some intermediate configurations were evaluated locally only.

References

- Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72.
- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2022. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*.
- Bryan Eikema and Wilker Aziz. 2022. Sampling-based minimum bayes risk decoding for neural machine translation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 10978–10993.
- Matthias Hagen, Maik Fröbe, Artur Jurk, and Martin Potthast. 2022. Clickbait spoiling via question answering and passage retrieval. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 7025–7036.
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *Proceedings of the International Conference on Learning Representations*.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Takeru Miyato, Andrew M Dai, and Ian Goodfellow. 2017. Adversarial training methods for semi-supervised text classification. In *Proceedings of the International Conference on Learning Representations*.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392.
- Elad Segal, Avia Efrat, Mor Shoham, Amir Globerson, and Jonathan Berant. 2020. A simple and effective model for answering multi-span questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 3074–3080.